

More first-order syntax

Professor Cian Dorr

9th October 2022

New York University

Relational first-order languages (review)

A relational signature is a set of strings (not containing special characters, not variables), together with an arity function assigning an arity to each of them, e.g.:

$$[\text{Loves} \mapsto 2, \text{Wise} \mapsto 1]$$

Call it Σ . Here are some members of $\mathcal{L}(\Sigma)$, the set of first order formulae over Σ :

$$\text{Wise}(x) \wedge (\text{Loves}(x, y) \wedge \neg \text{Loves}(y, x''))$$

$$\exists x \exists y (\text{Wise}(x) \wedge \text{Loves}(x, y))$$

$$(\text{Wise}(x) \wedge \exists x \text{Wise}(x))$$

Unofficially:

$$\text{Wise}(x) \text{Loves}(x, y) \text{Loves}(y, x''')$$

$$xy(\text{Wise}(x) \text{Loves}(x, y))$$

$$\text{Wise}(x) x \text{Wise}(x)$$

A perspective on unofficial notation

Sample recursive definitions for relational formulae

Definition

FV is the function $\mathcal{L}(\Sigma) \rightarrow \mathcal{P}(\text{Var})$ such that

1. For any atomic formula $P = R(v_1, \dots, v_n)$, $FV(P) = \{v_1, \dots, v_n\}$.
2. For any formula P , $FV(P) = FV(P)$.
3. For any formulae P, Q , $FV(PQ) = FV(PQ) = FV(PQ) = FV(P)FV(Q)$
4. For any formula P and variable v , $FV(vP) = FV(vP) = FV(P) \setminus \{v\}$.

We say that P is *closed*, (a *sentence* of $\mathcal{L}(\Sigma)$) when $FV(P) = \emptyset$; otherwise *open*.

Intuitively: if we have a particular way of *understanding* $\mathcal{L}(\Sigma)$, the sentences are going to be the ones that we can use to *assert that something is the case*. *Open* formulae by contrast are more analogous to *predicates* in English; e.g. $\text{loves}(x, x) \approx$ 'loves itself'.

Capture-free substitution

Example

For any variables u, v , $[u/v] : \mathcal{L}(\Sigma) \rightarrow \mathcal{L}(\Sigma)$ is the largest partial function such that:

$$(1) \quad R(v_1, \dots, v_n)[u/v] = R(u_1, \dots, u_n) \text{ where } u_i = v_i \text{ if } v_i \neq x \text{ and } u_i = y \text{ if } v_i = x$$

$$(2) \quad (P)[u/v] = (P[u/v])$$

$$(3) \quad (P \# Q)[u/v] = P[u/v] \# Q[u/v] \text{ for } \# = , ,$$

$$(4) \quad (wP)[u/v] = \begin{cases} wP & \text{if } v = w \\ w(P[u/v]) & \text{if } vw \text{ and } (uw \text{ or } v \notin FV[P]) \\ \text{undefined} & \text{if } vw \text{ and } u = w \text{ and } v \in FV[P] \end{cases}$$

$$(5) \quad (wP)[u/v] = \begin{cases} wP & \text{if } v = w \\ w(P[u/v]) & \text{if } vw \text{ and } uw \text{ or } v \notin FV[P] \\ \text{undefined} & \text{if } vw \text{ and } u = w \text{ and } v \in FV[P] \end{cases}$$

Capture-free substitution

The function defined on the previous slide is called *capture-free substitution of u for v* .

Its definition depends on a minor extension of the recursion theorem which allows for partial functions. We can recover this from our recursion theorem by defining it as a total function on $\mathcal{L}() \cup \{() \text{undefined}()\}$, then restricting to $\mathcal{L}()$.

- Why do we care about this rather messy function? The answer is that we are going to want to use the following rules to capture reasoning about quantifiers:

$$E : P[u/v] \text{ follows from } vP$$
$$I : vP \text{ follows from } P[u/v]$$

But $x\text{Loves}(x, x)$ should not follow from $yx\text{Loves}(x, y)$!

Also, $\text{Wise}(x)y\text{Loves}(x, x)$ should not follow from $y(\text{Wise } yy\text{Loves}(y, y))$.

Adding function symbols

First-order signatures

We are going to be interested in first-order languages that are suited for talking about *numbers* and *strings*.

One could formalize theories in arithmetic using a relational signature. For example, we could talk about addition using a three-place predicate *Sum* where $\text{Sum}(x, y, z)$ means ‘ x and y add up to z ’. We could express the commutativity of addition using a sentence

$$x_1 x_2 y (\text{Sum}(x_1, x_2, y) \text{ Sum}(x_2, x_1, y))$$

and its associativity with

$$x_1 x_2 x_3 z (y (\text{Sum}(x_1, x_2, y) \text{ Sum}(y, x_3, z)) y (\text{Sum}(x_2, x_3, y) \text{ Sum}(x_1, y, z)))$$

But this gets really awkward.

First-order signatures

For such purposes, it's a lot more convenient to use a more general kind of first-order language that contains *function symbols* as well as predicates. If we talk about addition with a binary function symbol $+$, we can express commutativity and associativity as one would expect, as:

$$xy(x + y = y + x)$$

$$xyz(x + (y + z) = (x + y) + z)$$

First-order signatures

Definition

A *first-order signature* is a triple $\Sigma = \langle R_\Sigma, F_\Sigma, a_\Sigma \rangle$, where R_Σ and F_Σ are two *non-overlapping* sets of strings that don't contain any special characters and are not in Var , and $a_\Sigma : (RF) \rightarrow \mathbb{N}$.

When $\Sigma = \langle R_\Sigma, F_\Sigma, a_\Sigma \rangle$ is a first-order signature, i is called a *predicate* of Σ if $i \in R_\Sigma$ and $a_\Sigma i > 0$; a *function symbol* of Σ if $i \in F_\Sigma$ and $a_\Sigma i > 0$; a *sentence constant* of Σ if $i \in R_\Sigma$ and $a_\Sigma i = 0$; and an *individual constant* of Σ if $i \in F_\Sigma$ and $a_\Sigma i = 0$.

Example

The *language of arithmetic* is the signature Arith with $R_{\text{Arith}} = \{\leq\}$, $F_{\text{Arith}} = \{0, \text{suc}, +, \times\}$, $a(\leq) = 2$, $a(0) = 0$, $a(\text{suc}) = 1$, $a(+) = 2$, $a(\times) = 2$.

First order languages with function symbols

Introducing individual constants and functional symbols makes the definition of *atomic formula* more complicated: we first define a different set of strings $\text{Terms}(\Sigma)$, and use it to define $\text{Atoms}(\Sigma)$.

Definition

When Σ is a first-order signature, $\text{Terms}(\Sigma)$ is the smallest set X of strings such that:

1. $v \in X$ for all $v \in \text{Var}$.
2. $c \in X$ whenever c is an individual constant of Σ .
3. $f(t_1, \dots, t_n) \in X$ whenever f is a function symbol of Σ with $a_\Sigma f = n$, and $t_1, \dots, t_n \in X$.

Unique readability for terms

Notice that we use parentheses and commas to demarcate a function symbol and its arguments. Again, this is to avoid ambiguity and secure

Unique Readability for Terms

The set $\text{Terms}(\Sigma)$, considered as an algebraic structure, has the Injective Property.

That is: the functions $([\langle t_1, \dots, t_n \rangle \mapsto f(t_1, \dots, t_n)])_{f \in F_\Sigma}$ are all injective, and their ranges do not overlap one another or Var or the set of individual constants of Σ .

This can be proved in exactly the same way as unique readability for relational formulae, by showing that every term is balanced and every initial substring of a term is either parenthesis-free or left-heavy.

Atomic formulae

We need new definitions of 'atomic formula' and 'formula' that allows terms that aren't variables to occur as arguments of predicates:

Definition

Where Σ is a first-order signature, $\text{Atoms}(\Sigma)$ is the set of all strings of the form

$$i \oplus (\oplus t_1 \oplus , \oplus \cdots \oplus t_n \oplus)$$

where each $t_i \in \text{Terms}()$ and either $i \in R_\Sigma$ and $n = a_\Sigma i$, or $i = =$ and $n = 2$.

The definition of 'formula' for first-order signatures is exactly the same as for relational signatures, just plugging in the new definition of atomic formula.

Unique readability

We can prove the following using the same techniques as before

Unique Readability for Atomic Formulae

For any signature Σ , any predicates $F, G \in R_\Sigma \cup \{=\}$ such that $a_\Sigma(F) = n$ or $F = =$ and $n = 2$ and $a_\Sigma(G) = m$, or $G = =$ and $m = 2$, and any $t_1, \dots, t_n, t'_1, \dots, t'_m \in \text{Terms}(\Sigma)$, if $F(t_1, \dots, t_n) = G(t'_1, \dots, t'_m)$ then $F = G$ and $n = m$ and $t_k = t'_k$ for each k .

Unique Readability Theorem for First-Order Formulae

When Σ is a first-order signature, $\mathcal{L}(\Sigma)$ (considered as an algebraic structure) has the Injective Property. That is, on the set of formulae of $\mathcal{L}(\Sigma)$, the functions $[s \mapsto \neg s]$, $[\langle s, t \rangle \mapsto (s \wedge t)]$, $[\langle s, t \rangle \mapsto (s \vee t)]$, $[\langle s, t \rangle \mapsto (s \sqcap t)]$, $([s \mapsto \sqcup v] s)_{v \in \text{Var}}$ and $([s \mapsto \sqcap v] s)_{v \in \text{Var}}$ are all injective, and their ranges do not overlap each other or $\text{Atoms}(\Sigma)$.

Recursive definitions for terms

Since $\text{Terms}(\Sigma)$ has the Inductive and Injective properties, we have a version of the Recursion Theorem for terms, and can use it to give recursive definitions such as the following.

Definition

Vars is the function $\text{Terms}(\Sigma) \rightarrow \mathcal{P} \text{Var}$ such that:

1. For any variable v , $\text{Vars}(v) = \{v\}$.
2. For any individual constant c , $\text{Vars}(c) = \emptyset$.
3. For any n -ary function symbol f and terms $t_1, \dots, t_n$,
 $\text{Vars}(f(t_1, \dots, t_n)) = \text{Vars}(t_1) \cup \dots \cup \text{Vars}(t_n)$.

Definition

For any variable v and $t \in \text{Terms}(\Sigma)$, $[t/v]$ is the unique function $\text{Terms}(\Sigma) \rightarrow \text{Terms}(\Sigma)$ such that

1. $v[t/v] = t$
2. $u[t/v] = u$ when u is a variable other than v .
3. $c[t/v] = c$ when c is an individual constant.
4. $f(t_1, \dots, t_n)[t/v] = f(t_1[t/v], \dots, t_n[t/v])$.

(Again we follow the tradition of writing this sort of function in postfix position.)

Recursive definitions for formulae, redux

A common pattern for defining a function from the set of formulae of a first-order signature Σ to some domain is to first recursively define a helper function from terms, then use it to define a function for atomic formulae, and finally recursively extend that to all formulae. For example:

Definition

FV is the function $\mathcal{L}() \rightarrow \mathcal{P}\text{Var}$ such that

1. For an atomic formula $P = R(t_1, \dots, t_n)$, $FV(P) = \text{Vars } t_1 \cup \dots \cup \text{Vars } t_n$.
2. For any formula P , $FV(P) = FV(P)$.
3. For any formulae P, Q , $FV(PQ) = FV(PQ) = FV(PQ) = FV(P)FV(Q)$
4. For any formula P and variable v , $FV(vP) = FV(vP) = FV(P) \setminus \{v\}$.

Definition

For any term t and variable v , $[t/v] : \mathcal{L}(\Sigma) \rightarrow \mathcal{L}(\Sigma)$ is the partial function such that:

$$R(v_1, \dots, v_n)[t/v] = R(t_1, \dots, t_n) \text{ where } t_i = v_i \text{ if } v_i \neq v \text{ and } t_i = t \text{ if } v_i = v$$

$$(P)[t/v] = (P[t/v])$$

$$(P \# Q)[t/v] = P[t/v] \# Q[t/v] \text{ for } \# = , ,$$

$$(uP)[t/v] = \begin{cases} uP & \text{if } v = u \\ u(P[t/v]) & \text{if } vu \text{ and } (u \notin FV(t) \text{ or } v \notin FV[P]) \\ \text{undefined} & \text{if } vu \text{ and } u \in FV(t) \text{ and } v \in FV[P] \end{cases}$$

$$(uP)[t/v] = \begin{cases} uP & \text{if } v = u \\ u(P[t/v]) & \text{if } vu \text{ and } (u \notin FV(t) \text{ or } v \notin FV[P]) \\ \text{undefined} & \text{if } vu \text{ and } u \in FV(t) \text{ and } v \in FV[P] \end{cases}$$

First-order languages with definite descriptions

Note: in a couple of weeks, we will introduce one more convenient expansion of first-order syntax, namely the addition of a *definite description operator*.

This is not crucial (any more than it's crucial to have function symbols rather than predicates), but will help simplify some definitions relevant to the “definability” and “representability” of functions.